THE OPENCODE PLAYBOOK FOR TEAMS

AI에게 질문을 넘어, 일을 맡기는 방법.

계획하고, 맡기고, 검증하고, 기록하세요.

OpenCode로 만드는 우리 팀의 새로운 작업 방식.

11개 챕터 • 약 30분 • 입문부터 팀 활용까지

◎ **목표는 적은 토큰이 아니라, 적은 재작업입니다.**

필요한 분석과 검증은 유지하고, 팀의 경험을 다음 작업에 연결하세요.

LEARN → PRACTICE → REPEAT

작은 작업부터, 끝까지.

CHAPTER 01 사내 경험

01 사내 OpenCode를 쓰는 이유

사내 업무 데이터는 승인된 사내 서비스와 모델에서 처리한다.

↗ 개인 토큰 잔량보다 작업 품질과 완료 조건에 집중한다.

↗ 혼잡 시간에는 실행량을 늘리기보다 다음 작업을 준비한다.

ⓘ **시작하기 전에, 사내 환경부터 확인하세요.**

승인 모델 · 설치 버전 · 할당량 · 지원 담당자: 사내 담당자 입력 필요

운영 경험: 14–22시 혼잡, 특히 15–17시 불안정 · 발표자 제공, 2026.10.01

본문

사내 코드와 문서를 다루려면 회사가 승인한 AI 서비스를 사용해야 합니다. OpenCode를 사내 모델에 연결하면, 코드를 읽고 수정하는 작업을 익숙한 개발 환경에서 진행할 수 있습니다. 단, **OpenCode를 설치했다는 사실만으로 데이터가 사내에 머무는 것은 아닙니다.** 연결된 모델과 플러그인, MCP, 브라우저 요청, 로그 저장까지 사내 정책을 따라야 합니다. [S01, S02]

우리가 원하는 것은 '토큰의 노예에서 탈출'하는 것입니다. 토큰이 모자랄까 봐 분석과 검증을 생략하는 대신, 승인된 자원 안에서 일을 끝내는 데 집중합니다. 사내 서비스의 할당량과 공유 자원 제약은 여전히 존재하므로 무제한 사용을 뜻하지는 않습니다.

사내 운영 경험

시간	발표자가 관찰한 상태	작업 방식 제안
14:00~22:00	응답 지연·중단이 발생하는 혼잡 구간	작업을 작게 나누고 중간 기록을 남긴다
특히 15:00~17:00	불안정이 두드러짐	프롬프트·계획·검증 조건을 먼저 정리한다
상대적으로 여유로운 시간	상황에 따라 실행하기 편함	준비된 작업을 실행하고 결과를 검토한다

이 시간표는 **발표자 제공 경험**입니다. 고정 성능이나 서비스 보장이 아니며, 운영 상황에 따라 달라질 수 있습니다. 실패한 요청을 동시에 계속 늘리면 혼잡이 악화될 수 있으므로 상태를 확인하고 재개합니다.

사내 사용 시 먼저 확인할 것

승인된 모델 연결인지, 외부 제공자로 자동 전환되는 설정은 없는지, 플러그인과 MCP가 허용되는지 확인합니다. 사내 세션의 공개 공유는 사용하지 않습니다. 공유 비활성화 설정은 설치판의 스키마를 확인한 후 적용합니다. [S01, S02]

```json
{
  "$schema": "https://opencode.ai/config.json",
  "share": "disabled"
}
```

이 설정 하나가 외부 통신 전체를 차단하지는 않습니다. 실제 차단은 회사의 네트워크·도구 정책과 함께 적용해야 합니다.

관련 출처

OpenCode Enterprise ↗ (https://opencode.ai/docs/enterprise/)

OpenCode Share ↗ (https://opencode.ai/docs/share/)

CHAPTER 02 공식 기능

02 OpenCode 첫 사용

OpenCode는 모델을 연결해 개발 작업을 수행하는 AI 코딩 에이전트다.

- ↗ 채팅뿐 아니라 파일 읽기·수정·명령 실행·작업 위임이 가능하다.
- ↗ 먼저 파일 지정, 계획/구현 전환, 세션 관리를 익힌다.

QUICK REFERENCE

필요한 명령어를 바로 찾으세요.

18개 명령어

원본 dev는 사내 설치판과 다를 수 있습니다. 기존 OmO는 호환 이해용입니다.

명령어	하는 일	구분 / 단축키
`opencode`	현재 프로젝트에서 시작	OpenCode 기본 터미널에서 입력
`/help`	도움말	OpenCode 기본 사용 가능한 기능 확인
`@파일경로`	파일을 대화 맥락에 추가	OpenCode 기본 필요한 파일만 지정
`!명령어`	셸 명령 실행	OpenCode 기본 결과가 대화에 들어감
`Tab`	기본 primary agent 전환	OpenCode 기본 Build / Plan 등 설치판의 역할
`/models`	모델 선택	OpenCode 기본 Ctrl+X → M
`/init`	AGENTS.md 생성·갱신 도움	OpenCode 기본 생성 결과를 검토
`/new`	새 세션 시작	OpenCode 기본 Ctrl+X → N
`/sessions`	이전 세션 조회·전환	OpenCode 기본 Ctrl+X → L
`/compact`	긴 대화 요약	OpenCode 기본 Ctrl+X → C

명령어	하는 일	구분 / 단축키
`/undo`	마지막 메시지와 관련 변경 되돌리기	OpenCode 기본 Ctrl+X → U, Git 필요
`/redo`	되돌린 작업 재적용	OpenCode 기본 Ctrl+X → R
`/editor`	긴 지시문을 외부 편집기로 작성	OpenCode 기본 Ctrl+X → E
`Ctrl+P`	명령 목록	OpenCode 기본 명령을 기억하지 못할 때 사용
`/start-work`	계획 실행 시작 · 기존 OmO 참고	기존 OmO 사내 설치판 확인 필요
`/ulw-plan`	계획 작성 · 원본 dev 문서 기준	원본 dev 사내 설치판 확인 필요
`/ulw-execute`	계획 실행 · 원본 dev 문서 기준	원본 dev 사내 설치판 확인 필요
`ulw / ultrawork`	탐색·분해·위임·검증 흐름	원본 dev 사내 설치판 확인 필요

명령어는 참고용입니다. 이 페이지에서 실행되지 않습니다.

본문

OpenCode는 프로젝트 안에서 AI 모델과 도구를 함께 사용하는 개발 환경입니다. 모델이 답을 생성하고, OpenCode가 파일·터미널·권한·세션을 연결합니다. 모델은 두뇌, OpenCode는 작업 공간으로 생각하면 이해하기 쉽습니다. 터미널 UI를 중심으로 사용할 수 있으며 제공 인터페이스는 설치판을 확인합니다. [S03, S04]

Oh My OpenCode는 기본 OpenCode와 별개의 확장입니다. 스킬은 반복 가능한 작업 절차이고, MCP는 외부 도구나 데이터와 연결하는 인터페이스입니다. 모두 같은 종류의 기능은 아닙니다.

먼저 익힐 명령과 단축키

다음은 확인 시점 OpenCode 기본 TUI 문서 기준입니다. 단축키는 사용자 설정으로 바뀔 수 있습니다. `Ctrl+X → N`은 동시에 누르는 조합이 아니라 Ctrl+X를 누른 다음 N을 누른다는 뜻입니다. [S03, S05]

입력	기능	기본 단축키 / 설명
`opencode`	현재 프로젝트에서 시작	터미널에서 입력
`/help`	도움말	사용 가능한 기능 확인

입력	기능	기본 단축키 / 설명
`@파일경로`	파일을 대화 맥락에 추가	필요한 파일만 지정
`!명령어`	셸 명령 실행	결과가 대화에 들어감
`Tab`	기본 primary agent 전환	Build / Plan 등 설치판의 역할
`/models`	모델 선택	Ctrl+X → M
`/init`	AGENTS.md 생성·갱신 도움	생성 결과를 검토
`/new`	새 세션 시작	Ctrl+X → N
`/sessions`	이전 세션 조회·전환	Ctrl+X → L
`/compact`	긴 대화 요약	Ctrl+X → C
`/undo`	마지막 메시지와 관련 변경 되돌리기	Ctrl+X → U, Git 필요
`/redo`	되돌린 작업 재적용	Ctrl+X → R
`/editor`	긴 지시문을 외부 편집기로 작성	Ctrl+X → E
`Ctrl+P`	명령 목록	명령을 기억하지 못할 때 사용

`/compact`는 대화 정리 기능입니다. 프로젝트 규칙과 진행 기록을 별도 파일로 남기는 일을 대신하지 않습니다.

반복 지시는 사용자 명령으로

예를 들어 '변경 내용을 읽고 리뷰하라'는 지시를 사용자 명령으로 저장하면 반복 입력을 줄일 수 있습니다. 프로젝트 명령은 `.opencode/commands/` 아래 Markdown 파일로 정의할 수 있습니다. 실제 문법은 설치판 문서를 따릅니다. [S06]

교육용 `review-change.md` 내용:

```markdown
---
description: 변경 사항 검토
---
현재 변경 내용을 읽고 오류 가능성, 회귀 위험, 필요한 검증을 정리하라.
파일은 수정하지 말고, 발견 사항마다 근거와 위치를 제공하라.
```

첫 작업 예시

PROMPT / 교육용 예제

```
이 프로젝트의 실행 방법과 테스트 방법을 확인해줘.
수정하지 말고 핵심 디렉터리, 진입점, 사내 규칙을 정리해줘.
확인하지 못한 것은 추측하지 말고 미확인이라고 표시해줘.
```

관련 출처

- OpenCode TUI ↗ (https://opencode.ai/docs/tui/)
- OpenCode Agents ↗ (https://opencode.ai/docs/agents/)
- OpenCode Keybinds ↗ (https://opencode.ai/docs/keybinds/)
- OpenCode Commands ↗ (https://opencode.ai/docs/commands/)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 03 교육 예제

03 스킬·하네스·메모리

#

같은 모델도 주어진 절차와 검증 환경에 따라 결과가 달라진다.

- ↗ 스킬은 작업 방법, 하네스는 실행·검증·재개를 관리하는 구조다.
- ↗ 반복 실수는 기억해 달라는 말보다 기록과 회귀 검증으로 줄인다.

A · 결과만 요구

“이 기능 만들어줘.”

기능의 모양은 있지만,
완료를 판단할 기준이 없습니다.

장비 색상과 클릭 정보창을 만들어줘.

B · 절차와 검증을 함께

“이 조건까지 확인해줘.”

스킬 → 계획 → 구현 → 검증 → 기록
다음 작업에도 쓸 수 있는 경험을 남깁니다.

선택 ID 일치 · 상태 갱신 · 회귀 검증

교육용 지시 비교 · 측정된 성능 비교가 아닙니다. 전체 지시문은 아래 원고에서 복사할 수 있습니다.

본문

OpenCode의 결과는 모델 능력만으로 결정되지 않습니다. 요구사항, 필요한 맥락, 도구, 스킬, 검증 방식이 함께 작동해야 합니다. ‘좋은 모델이면 알아서 한다’보다 **좋은 작업 환경을 만들어 준다**는 관점이 유용합니다.

요소	역할	교육용 예
모델	문제를 해석하고 판단	구현 방안 제안
스킬	특정 업무의 재사용 절차	Babylon.js 변경 시 picking·색상·성능 점검
하네스	실행·권한·검증·재개를 연결	테스트 실행, 실패 시 수정, 결과 기록
메모리	다음 작업에 필요한 규칙과 경험 보존	thin instance 식별 오류와 해결 조건 기록

OpenCode 스킬은 `SKILL.md`로 정의하고 필요할 때 로드하는 방식입니다. 장기 작업 하네스에 대한 참고자료로는 Anthropic의 운영 사례가 있습니다. 이 자료는 진행 기록과 실제 사용자 흐름 검증을 장기 작업에 연결하는 접근을 소개합니다. [S07, S09]

같은 과제를 다르게 맡기는 예제

과제: Babylon.js 장비 1,000개의 상태 색상과 클릭 정보창 구현.

A — 결과만 요구

PROMPT / 교육용 예제

```
장비 1,000개를 만들고 상태에 따라 색상을 바꿔줘.
클릭하면 장비 정보가 뜨게 해줘.
```

B — 절차·검증·기록까지 요구

PROMPT / 교육용 예제

```
장비 1,000개의 상태 색상과 클릭 정보창을 구현한다.
먼저 기존 장면 구조와 관련 팀 스킬을 읽고 구현 계획을 작성한다.
반복 장비의 형상과 재질 조건을 확인한 뒤 thin instance 적용 여부를 판단한다.
객체 식별과 picking 매핑을 점검하고, 색상 적용 방식은 현재 셰이더 구조에 맞춘다.

완료 조건:
- 정상/경고/오류 상태가 구분된다.
- 클릭한 장비의 ID와 정보창의 ID가 일치한다.
- 상태 갱신 후 색상이 반영된다.
- 기존 카메라 조작이 유지된다.

검증:
- ID 매핑과 상태 변환은 단위 테스트로 확인한다.
- Playwright로 클릭과 정보창을 확인하고 화면을 캡처한다.
- 성능은 지정된 기기와 장면 조건에서 측정하고 실제 값을 보고한다.
- 이번 오류와 재발 방지 방법을 진행 기록에 남긴다.
```

이 비교는 교육용 지시문이며 측정된 성능 비교가 아닙니다. 실습에서는 같은 모델·코드 시작점·과제를 사용하고 누락 기능, 테스트 결과, 재작업 횟수, 소요 시간을 기록합니다. B도 실패할 수 있으므로 실제 결과로 판단합니다.

반복 실수를 남기는 구조

PROMPT / 교육용 예제

```
AGENTS.md                  팀 공통 규칙과 시작 시 읽을 문서
docs/ai/known-pitfalls.md   검증된 반복 오류와 회피 방법
docs/ai/decisions.md        설계 결정과 이유
docs/ai/progress.md         현재 상태, 미완료, 다음 행동
tests/                     재발 여부를 확인할 검증
```

`docs/ai/*`는 **이 자료에서 제안한 팀 규약**입니다. 파일을 만들기만 하면 자동으로 읽힌다고 가정하지 않습니다. AGENTS.md나 작업 지시에서 필요한 기록을 읽도록 연결합니다. OpenCode는 프로젝트 규칙을 AGENTS.md로 제공할 수 있습니다. [S08]

좋은 오류 기록에는 '증상 → 원인 → 적용 조건 → 해결 → 재발 검증'이 들어갑니다. 임시 추측과 비밀번호를 영구 규칙에 넣지 않고, 오래된 내용을 정리합니다.

이것이 루프 엔지니어링인가?

메모리 파일 작성 자체는 주로 맥락·메모리 관리입니다. **실행 → 검증 → 오류 기록 → 절차 수정 → 재실행**까지 연결하면 반복 개선 루프를 설계하는 것으로 설명할 수 있습니다. '루프 엔지니어링'은 여기서 교육적 표현이며 OpenCode의 표준 기능명은 아닙니다.

반복에는 종료 조건도 필요합니다. 예를 들어 같은 오류가 두 차례 반복되면 사람에게 원인과 로그를 넘기고, 검증 도구를 실행할 수 없으면 완료로 표시하지 않습니다.

관련 출처

OpenCode Agent Skills ↗ (https://opencode.ai/docs/skills/)

OpenCode Rules ↗ (https://opencode.ai/docs/rules/)

Anthropic: Effective harnesses for long-running agents ↗ (https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 04 버전 확인 필요

04 Oh My OpenCode

#

확장은 탐색·분해·위임·검증 흐름을 구성하는 데 도움을 준다.

- ↗ ultrawork는 작업 방식이며, 별도의 초고성능 모델을 만드는 기능은 아니다.
- ↗ 간단한 수정은 직접, 복잡한 작업은 구조화해서 맡긴다.

본문

Oh My OpenCode는 OpenCode를 확장해 작업 운영을 돕는 프로젝트입니다. 원본 저장소는 확인 시점 Oh My OpenAgent로 연결됩니다. 사내에서 계속 Oh My OpenCode라는 이름을 사용하더라도 설치된 버전의 명령과 문서를 기준으로 해야 합니다. [S12]

`ultrawork` 또는 `ulw`는 작업을 적극적으로 탐색·분해·위임·검증하도록 유도하는 방식입니다. 'Ultra Worker를 만든다'는 표현은 이런 작업 흐름을 구성한다는 의미로 설명합니다. `Ultraworker`라는 고정 에이전트가 모든 버전에 존재하는 것은 아닙니다. [S13]

기대하는 이점과 적용 판단

- 관련 코드를 찾고 작업을 나누는 부담을 줄일 수 있습니다.
- 독립 작업을 위임하고 각 작업에 필요한 스킬을 연결할 수 있습니다.
- 완료 조건과 증거를 중심으로 결과를 검토하는 흐름을 만들 수 있습니다.

상황	제안
오타 수정·단일 설정 변경	기본 에이전트로 직접 처리
여러 파일에 걸친 기능 구현	계획을 문서화한 후 실행
작업 분해와 탐색이 많이 필요	설치판의 ultrawork 흐름 검토
같은 파일을 여러 작업이 수정	병렬화보다 책임 분리·통합 순서 우선
사내 서비스 혼잡	동시 실행 수를 줄이고 작업 준비

위 판단표는 팀 운영 제안입니다. 위임에는 추가 문맥·토큰·조정 비용이 들며, 항상 빠르거나 정확한 결과를 보장하지 않습니다.

교육용 지시:

PROMPT / 교육용 예제

```
ulw
이 작업의 목표는 사용자 목록 필터 기능 구현이다.
기존 구조를 확인하고 범위를 나눈 뒤 적절한 스킬과 검증 방법을 적용해줘.
완료 조건과 실제 검증 결과를 보고해줘.
사내에서 승인된 모델과 도구만 사용해줘.
```

관련 출처

OmO 원본 저장소 ↗ (https://github.com/code-yeongyu/oh-my-openagent)

OmO 원본 orchestration 문서, dev ↗ (https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/guide/orchestration.md)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 05 교육 예제

05 프롬프트 설계

#

챗봇으로 업무 지시를 정리한 다음 실행 도구에 전달한다.

↗ 목표·범위·제약·완료 조건·검증 방법을 빠뜨리지 않는다.

↗ 회사 정보는 승인된 챗봇에서만 처리한다.

본문

머릿속 아이디어를 바로 긴 작업으로 맡기면 실행 중에 조건이 계속 달라질 수 있습니다. ChatGPT나 다른 챗봇으로 요구사항을 먼저 정리하면, OpenCode에 전달할 지시를 더 명확하게 만들 수 있습니다. 이는 이 교육에서 권장하는 작업 방법이며 성능을 보장하는 제품 기능은 아닙니다.

외부 챗봇을 사용할 때는 공개 가능한 추상 과제나 가상 데이터로 정리합니다. 실제 사내 코드·엑셀·고객 정보를 다뤄야 한다면 승인된 사내 챗봇을 사용합니다.

1단계 — 챗봇에게 업무 지시를 정리시키기

PROMPT / 교육용 예제

```
다음 요구를 코딩 에이전트에 전달할 작업 지시로 정리해줘.
목표, 범위, 제외 범위, 제약, 완료 조건, 검증 방법, 산출물 순서로 작성해줘.
확인되지 않은 환경은 만들어내지 말고 확인 필요로 표시해줘.
구현 방법을 과하게 고정하지 말고, 기존 프로젝트 구조를 먼저 확인하도록 해줘.

요구: 장비 목록에 상태 필터를 추가하고, 선택한 장비를 3D 화면에서 강조하고 싶다.
```

2단계 — 실행할 프롬프트

PROMPT / 교육용 예제

```
목표: 장비 목록에서 상태를 필터링하고 선택한 장비를 3D 화면에서 강조한다.
범위: 목록 필터, 선택 이벤트, 장면의 강조 처리.
제외: 인증 변경, 전체 UI 개편, 관련 없는 리팩터링.
제약: 기존 구조와 팀 규칙을 따른다. 승인된 도구만 사용한다.
완료 조건: 필터와 표시 목록이 일치하고 선택 ID와 강조 장비 ID가 일치한다.
검증: 필터 단위 테스트와 실제 선택 흐름의 브라우저 테스트.
산출물: 변경 요약, 실제 검증 결과, 남은 문제, 재개 기록.

먼저 관련 코드를 확인하고 계획을 제시해줘. 계획 확인 후 구현한다.
```

좋은 프롬프트는 길이가 아니라 판단 기준이 명확합니다. 필요한 맥락을 유지하면서 중복 문장을 줄입니다.

관련 출처

OpenCode TUI ↗ (https://opencode.ai/docs/tui/)

OpenCode Rules ↗ (https://opencode.ai/docs/rules/)

CHAPTER 06 버전 확인 필요

06 계획·실행·세션 관리

Planner는 무엇을 할지, Executor는 계획을 어떻게 진행할지 담당한다.

- ↗ 계획 파일과 대화 세션은 서로 다른 기록이다.
- ↗ 끊겼을 때는 현재 코드와 검증 기록을 확인한 뒤 재개한다.

역할 이해

역할	하는 일	주요 산출물
Planner	요구 분석, 범위·의존성·완료 조건 정리	작업 계획
Plan Executor / Orchestrator	계획 순서 관리, 위임, 결과 통합	진행 상태와 통합 결과
Worker	특정 구현·탐색·테스트 수행	변경 내용과 근거
Reviewer / QA	결과와 완료 조건을 독립적으로 확인	발견 사항과 검증 증거
사람	우선순위·제약·수용 여부 판단	승인된 목표와 다음 작업

이 표는 역할 개념입니다. 역할을 나눈다고 반드시 서로 다른 모델이나 별도 프로세스가 필요한 것은 아닙니다. OpenCode 기본 Plan·Build와 플러그인의 역할은 따로 이해합니다. [S04]

설치 버전에 따른 차이

구분	기존 OmO 자료에서 알려진 방식	현재 원본 dev 문서에서 확인한 방식
계획·실행	Prometheus / Atlas	메인 에이전트의 계획·실행 흐름
명령 예	계획 후 `/start-work`	`/ulw-plan` 후 `/ulw-execute`
계획 경로	`.sisyphus/plans/*.md`	`.omo/plans/*.md`
진행 상태 경로	`.sisyphus/boulder.json`	`.omo/boulder.json`

기존 열은 호환 이해용입니다. 사내 설치판에서 해당 이름·파일·명령이 있는지 확인한 뒤 실습에 사용합니다. 현재 열도 최신 안정 릴리스라는 뜻은 아닙니다. 원본 dev 문서 기준이며 발표 전에 버전을 고정해야 합니다. [S13, S14]

계획은 무엇을 저장하는가

계획 파일에는 작업 항목, 참조 파일, 완료 조건, 검증 방법을 남깁니다. 진행 상태는 어떤 계획을 실행 중인지 연결하고, 대화 세션은 요청과 응답을 보존합니다. **계획 파일만으로 이전 대화의 모든 맥락이 복원되지는 않습니다.**

기본 Plan에서 만든 설명이 언제나 특정 경로에 자동 저장된다고 가정하지 않습니다. 플러그인을 쓰지 않는 팀은 계획을 `docs/ai/plans/<작업명>.md`에 저장하도록 명시할 수 있습니다. 이 경로는 팀 제안이며 OpenCode 기본 저장 경로가 아닙니다.

재개할 때의 순서

1. `/sessions`에서 해당 대화를 찾거나 작업 계획을 명시해 새 세션을 시작합니다.
2. 해당 설치판의 계획·진행 기록과 실제 변경 내용을 읽습니다.
3. 마지막 테스트 결과를 확인하고 필요한 검증을 다시 실행합니다.
4. 완료된 항목과 남은 항목을 맞춘 뒤 다음 작업을 진행합니다.

진행 파일을 지워서 재개 문제를 해결하려고 하기 전에 백업과 원인을 확인합니다. 새 세션 시작과 진행 상태 삭제는 다른 행동입니다.

복사할 재개 지시:

PROMPT / 교육용 예제

```
이 작업의 계획 파일과 진행 기록을 읽고 재개해줘.
먼저 실제 코드 변경과 마지막 검증 결과가 기록과 일치하는지 확인해줘.
완료/미완료/막힌 항목을 정리한 뒤 다음 미완료 항목을 수행해줘.
이전 대화에만 있고 파일에 없는 내용은 추측하지 말아줘.
```

관련 출처

OpenCode Agents ↗ (https://opencode.ai/docs/agents/)

OmO 원본 orchestration 문서, dev ↗ (https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/guide/orchestration.md)

기존 방식 참고용 보존 문서: mathpal/oh-my-opencode fork ↗ (https://github.com/mathpal/oh-my-opencode/blob/dev/docs/guide/orchestration.md)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 07 교육 예제

07 나만의 에이전트 팀

#

탐색·구현·검증을 서로 다른 역할로 나눌 수 있다.

↗ 독립 작업은 병렬화하고, 의존 작업은 순서를 지킨다.

↗ 동일 파일 수정과 결과 통합은 명확히 관리한다.

본문

OpenCode는 primary agent와 subagent를 구분하고, 사용자 정의 에이전트를 구성할 수 있습니다. 개인도 '구현 담당·검증 담당·리뷰 담당'처럼 역할을 정의할 수 있습니다. 실제 위임 방법과 권한 설정은 설치판을 확인합니다. [S04]

팀 구성은 사람 조직을 그대로 흉내 내는 데 목적이 있지 않습니다. 중요한 것은 **작업 경계와 결과를 받을 형식**입니다.

교육용 팀 구성:

담당	입력	맡기는 범위	결과
조정자	목표와 완료 조건	분해·순서·통합	진행표
탐색 담당	관련 기능 질문	읽기와 위치 확인	관련 파일과 위험
구현 담당	명확한 변경 범위	제품 코드 수정	변경 요약
테스트 담당	완료 조건	합의된 테스트 파일	실행 결과
리뷰 담당	변경 내역	읽기와 검토	문제와 근거

병렬로 진행할 수 있는 것

목록 필터 로직과 독립된 도움말 문구는 병렬로 작업할 수 있습니다. 반면 공통 데이터 구조 변경과 그 구조를 사용하는 기능 구현은 계약을 먼저 합의해야 합니다.

교육용 지시:

PROMPT / 교육용 예제

```
작업을 구현·검증·문서화로 나누어줘.
독립 작업만 병렬로 수행하고 동일 파일 수정은 겹치지 않게 해줘.
각 작업의 입력, 수정 가능 범위, 완료 조건, 결과 형식을 먼저 정리해줘.
마지막에는 변경 내용을 통합하고 전체 검증을 실행해줘.
```

필요하다면 worktree로 작업 공간을 분리하지만 통합 책임은 남습니다. 처음에는 작은 수의 역할로 시작하고, 공유 모델 서버의 혼잡과 위임 비용을 보면서 늘립니다.

관련 출처

OpenCode Agents ↗ (https://opencode.ai/docs/agents/)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 08 교육 예제

08 다음 일을 설계하는 법

#

사람은 목표·완료 조건을 준비하고 AI는 합의된 작업을 실행한다.

- ↗ 실행 중에는 독립된 다음 작업을 준비한다.
- ↗ 결과 검증과 기록이 끝난 뒤 다음 계획에 반영한다.



본문

업무는 '계획 → 일 시키기 → 기다리기'에서 끝나지 않습니다. 한 작업이 진행되는 동안 사람은 다음 과제의 조건과 검증 방법을 준비할 수 있습니다. 단, 진행 중인 코드와 같은 파일을 동시에 바꾸거나, 아직 확정되지 않은 결과에 의존해 다음 구현을 시작하지 않습니다.


01
목표
02
계획
03
실행
04
검증
05
기록
↳ 검증 실패 시 수정 후 다시 검증 · 기록을 다음 계획에 반영 ↲

실무 예

작업 A의 목록 필터 구현이 실행 중일 때, 작업 B의 엑셀 출력 항목과 가상 샘플을 정리합니다. A 결과가 도착하면 테스트와 변경 내용을 검토하고, 확정된 데이터 구조를 B 계획에 반영합니다.

시점	사람의 일	에이전트의 일
시작 전	목표·제약·완료 조건 정리	구조 탐색·계획
실행 중	독립된 다음 과제 준비	구현·중간 검증

시점	사람의 일	에이전트의 일
결과 도착	증거 확인·수용 판단	실패 수정·결과 정리
완료 후	다음 우선순위 확정	진행·오류 기록 갱신

혼잡 시간에는 다음 지시문을 다듬고 테스트 조건을 준비합니다. 야간 자동 실행은 별도 운영 설계가 필요한 기능이며 이 자료에서 자동으로 설정하지 않습니다.

관련 출처

Anthropic: Effective harnesses for long-running agents ↗ (https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 09 교육 예제

09 검증과 증거

#

로직은 unit test, 사용자 흐름은 Playwright로 확인한다.

- ↗ Trivy 같은 CLI 도구를 검증 단계에 연결할 수 있다.
- ↗ 도구가 검사한 범위와 실행 증거를 함께 보고한다.

검증 방법

방법	확인하는 것	보고할 증거
Unit test	함수·변환·경계값	실행 명령과 통과/실패
Playwright	클릭·입력·화면·전체 흐름	assertion, trace, 필요 시 화면
타입 검사·빌드	타입 오류와 빌드 가능 여부	실제 실행 결과
Trivy	지원 대상의 취약점·설정·시크릿 등	스캐너 설정과 결과
사람의 검토	요구 적합성·업무 판단·시각 품질	수용 또는 수정 의견

Playwright 예제

```ts
import { test, expect } from '@playwright/test';

test('경고 장비 선택 시 정보가 일치한다', async ({ page }) => {
  await page.goto('/equipment'); // baseURL 설정 필요
  await page.getByRole('button', { name: '경고만 보기' }).click();
  const row = page.getByTestId('equipment-E-102');
  await expect(row).toBeVisible();
  await row.click();
  await expect(page.getByTestId('equipment-info')).toContainText('E-102');
});
```

교육용 예시입니다. 프로젝트에는 해당 접근성 이름·test ID·가상 장비 데이터가 있어야 합니다. 화면을 찍는 것만으로 통과라고 판단하지 않고 기대 결과를 검증합니다. Playwright의 assertion은 화면 상태를 확인할 때 사용할 수 있습니다. [S10]

3D 장면에서는 목록과 정보창 검증 외에 실제 picking 매핑·강조 상태도 확인해야 합니다. 스크린샷은 보조 증거이며 GPU·브라우저 차이에 따른 시각 검토도 필요합니다.

Trivy 같은 실행 도구 연결

```sh
# 교육용 예: 설치와 DB 준비가 필요
trivy fs --scanners vuln,secret,misconfig --exit-code 1 .
```

이 예시는 발견 결과를 실패 종료 코드로 연결하는 방식입니다. 팀에서 허용할 심각도·예외·스캐너 범위는 별도로 정합니다. Trivy는 코드의 모든 악성 행위나 외부 전송을 증명하는 도구가 아니며, 스캔 통과만으로 보안을 보장할 수 없습니다.

Trivy는 실행 방식에 따라 DB 등 외부 자원에 접근할 수 있습니다. 사내 사용 시 내부 미러 또는 오프라인 DB를 준비하고 공식 네트워크 운용 문서를 확인합니다. [S11]

완료 보고 템플릿

PROMPT / 교육용 예제

```
변경 내용:
실행한 검증 명령:
통과/실패/미실행 결과:
스크린샷·로그·리포트 위치:
미해결 문제와 영향:
다음 작업을 위한 기록:
```

설치되지 않은 Playwright나 Trivy가 자동으로 실행된다고 기대하지 않습니다. 실행 도구·권한·의존성·테스트 데이터를 먼저 준비합니다.

관련 출처

Playwright Assertions ↗ (https://playwright.dev/docs/test-assertions)

Trivy Connectivity and Network considerations ↗ (https://trivy.dev/docs/latest/advanced/air-gap/)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 10 사내 확인 필요

10 보안 엑셀 파일

#

승인된 환경에서 파일 접근과 모델 전달 범위를 구분한다.

- ↗ 원본을 보호하고 필요한 항목만 다룬다.
- ↗ 결과의 숫자·수식·서식을 별도로 확인한다.

▤ **엑셀 실습 안내는 준비 중입니다.**

허용 도구, 파일 준비, 문서보안 승인 절차, 가상 실습 파일을 사내 담당자가 추가해야 합니다.

본문 — 개략 안내

엑셀 파일 작업은 '파일을 읽는 프로그램'과 '읽은 내용을 받는 모델'을 함께 봐야 합니다. 로컬에서 Python으로 읽어도 그 내용을 외부 모델 프롬프트로 보내면 데이터 이동이 발생합니다. 외부 전송 여부는 파일 위치만으로 판단할 수 없습니다.

교육에서는 가상 엑셀로 구조를 설명합니다. 실제 보안 파일은 승인된 사내 환경에서만 사용하고 원본 대신 사본을 작업 대상으로 합니다. 필요한 시트와 열을 제한하고, 개인정보와 불필요한 값이 로그·프롬프트·캡처에 들어가지 않도록 합니다.

개략 순서:

1. 파일 등급과 접근 권한 확인.
2. 작업 사본 준비, 필요한 시트·열 지정.
3. 승인된 도구로 읽고 처리.
4. 행 수·합계·수식·서식·결과 파일 검증.
5. 사내 정책에 따라 중간 파일과 로그 관리.

암호·문서보안·DRM이 걸린 파일은 승인된 접근 방법을 따릅니다. 에이전트가 보호를 우회하도록 지시하지 않습니다. 파일 형식에 따라 수식 계산, 매크로, 서식 보존의 지원 범위가 다를 수 있으므로 처리 도구를 확인합니다.

발표자가 보충할 영역

- 사내 파일을 준비하는 방법: 입력 필요.
- 허용된 도구와 읽기 범위: 입력 필요.
- 문서보안 적용 파일의 승인 절차: 입력 필요.
- 가상 파일 예제와 검증 방법: 발표자가 추가.

이 장은 의도적으로 개략 수준입니다. 특정 사내 보안 제품이나 우회 방법을 만들어 넣지 않습니다.

관련 출처

OpenCode Enterprise ↗ (https://opencode.ai/docs/enterprise/)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

CHAPTER 11 교육 예제

11 팀의 다음 행동

#

잘 만든 스킬과 하네스를 찾아 검토하고 팀 환경에 맞춘다.

↗ 불필요한 문맥은 줄이고 필요한 분석·검증은 유지한다.

↗ 목표는 적은 토큰이 아니라 적은 재작업과 검증된 결과다.

본문

OpenCode를 잘 쓰는 핵심은 일을 명확하게 정의하고, 적절한 스킬과 실행 환경을 연결하며, 결과를 검증하는 것입니다. 팀이 만든 공통 규칙과 검증 절차를 사용하면 매번 같은 지시를 다시 만드는 부담을 줄일 수 있습니다.

이미 잘 만들어진 스킬과 하네스가 있다면 활용합니다. 다만 우리 프로젝트·설치 버전·사내 정책에 맞는지 확인하고, 실행 스크립트와 외부 접근 여부도 검토합니다. 그대로 설치하는 것과 검토해서 재사용하는 것은 다른 과정입니다.

'토큰을 아껴 써'라는 지시만으로 작업 품질이 좋아지지는 않습니다. 필요한 탐색·분석·검증까지 줄이면 품질이 떨어지고 재작업이 늘 수 있습니다. 중복 로그와 불필요한 맥락은 줄이되, 완료 조건을 확인하는 데 필요한 작업은 유지합니다.

이는 모든 상황에서 토큰 절약이 성능을 떨어뜨린다는 주장이 아닙니다. 우리가 줄이려는 것은 낭비이고, 지키려는 것은 검증된 결과입니다.

내일부터 해볼 작은 실습

- [ ] 승인된 모델과 설치 버전을 확인한다.
- [ ] 작은 기능 하나의 목표와 완료 조건을 작성한다.
- [ ] 관련 규칙과 스킬을 읽고 계획을 만든다.
- [ ] 일을 맡기고 실행 중에는 독립된 다음 과제를 준비한다.
- [ ] 실제 테스트 결과를 보고 완료를 판단한다.
- [ ] 오류와 다음 행동을 기록하고 다음 세션에서 읽는다.

마지막 화면 문구

AI에게 질문만 하지 말고, 일을 설계하고 맡기고 검증하세요.

좋은 스킬과 하네스는 팀의 경험을 다음 작업에 연결합니다.

관련 출처

OpenCode Agent Skills ↗ (https://opencode.ai/docs/skills/)

OpenCode Rules ↗ (https://opencode.ai/docs/rules/)

제공 자료 확인일 2026.10.01 · 설치 버전 대조 필요

KEEP IT HANDY

용어와 팀 공통 템플릿

부록 A. 용어집

용어	설명
Agent	모델과 도구를 사용해 작업을 수행하는 주체
Skill	특정 업무에 사용할 재사용 절차·참고자료
Harness	실행·검증·권한·재개를 관리하는 주변 구조

용어	설명
MCP	도구·데이터와 연결하는 인터페이스
Session	요청·응답·도구 사용이 이어지는 대화 단위
Plan	목표를 완료하기 위한 작업·검증 목록
Orchestrator	작업 분해·위임·결과 통합을 관리하는 역할
Memory	다음 작업에 활용할 규칙·결정·경험 기록
Regression test	이전에 되던 기능이나 해결한 오류가 다시 깨지는지 확인하는 검증

부록 B. 팀 공통 작업 지시

PROMPT / 교육용 예제

```
목표:
범위:
제외 범위:
현재 환경과 참조 파일:
사내 제약:
완료 조건:
실행할 검증:
결과 보고 형식:
계획·진행 기록 경로:
```

부록 C. 발표 진행 제안

도입과 첫 사용 5분 → 스킬·하네스 A/B 예제 5분 → 프롬프트·역할·세션 7분 → 업무 반복·검증 7분 → 엑셀 보충·결론 6분. 총 약 30분이며 실제 시연 시간에 따라 조정합니다.

REFERENCES

출처 및 참고자료

제공 자료 확인일: 2026.10.01. 원본 dev 문서는 안정 릴리스와 다를 수 있습니다.

사내 게이트웨이와 승인 환경

(https://opencode.ai/docs/enterprise/)

S02

OpenCode Share ↗

공개 공유 및 비활성화

(https://opencode.ai/docs/share/)

S03

OpenCode TUI ↗

기본 명령, 파일·셸 참조

(https://opencode.ai/docs/tui/)

S04

OpenCode Agents ↗

Plan/Build, primary/subagent

(https://opencode.ai/docs/agents/)

S05

OpenCode Keybinds ↗

기본 키와 사용자 설정

(https://opencode.ai/docs/keybinds/)

S06

OpenCode Commands ↗

사용자 명령 정의

(https://opencode.ai/docs/commands/)

S07

OpenCode Agent Skills ↗

SKILL.md와 필요 시 로드

(https://opencode.ai/docs/skills/)

S08

OpenCode Rules ↗

AGENTS.md 프로젝트 규칙

(https://opencode.ai/docs/rules/)

S09

Anthropic: Effective harnesses for long-running agents ↗

장기 작업 기록·검증의 참고 사례

(https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents)

S10

Playwright Assertions ↗

사용자 흐름 assertion

(https://playwright.dev/docs/test-assertions)

S11

Trivy Connectivity and Network considerations ↗

내부 미러·오프라인 운용

(https://trivy.dev/docs/latest/advanced/air-gap/)

S12

OmO 원본 저장소 ↗

이름 변경 및 배포판 확인

(https://github.com/code-yeongyu/oh-my-openagent)

S13

OmO 원본 orchestration 문서, dev ↗

현재 계획·실행 개념과 경로

(https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/guide/orchestration.md)

S14

기존 방식 참고용 보존 문서: **mathpal/oh-my-opencode fork ↗**

Prometheus/Atlas·sisyphus 방식의 역사적 참고

(https://github.com/mathpal/oh-my-opencode/blob/dev/docs/guide/orchestration.md)

발표 전 사내 담당자 입력 필요

OpenCode 버전, OmO 버전·배포판, 승인 모델 ID, 설치 안내, 보안 지침, 할당량, 최신 혼잡 상태, 지원 담당자, 엑셀 실습 방법.

opencode

TEAM PLAYBOOK · 2026

일을 설계하고, 맡기고, 검증하세요.